

DD2459 Software Reliability 2025
Take-home Exam
Tuesday 11 March, 15.00 – Thursday 13 March, 12.00 midday
(Version 1.0)

Instructions.

Please read the following instructions carefully.

- (1) Only hard-copy exam submissions are accepted. Clearly mark at the top of each sheet of paper you use: (a) your name, (b) the page number.
- (2) On your front page indicate: (a) how many pages are contained in your work in total, (b) your name (c) your personal or KTH number, (d) your e-mail address (in case I need to contact you)
- (3) There are two ways to **submit** your work. (a) Your work may be **handed in personally by you only** to EECS studentexpedition, E building, level 4, Lindstedsvägen 3, no later than Thursday 13th March 2025 at 12.00 midday. After this time, it will be marked as late, and marks will be subtracted. **Please take identification with you if you submit this way, e.g. a legitimization or passport.** (b) If you are unable to reach the studentexpedition yourself (for example if you are at work) you may **post** your manuscript to: Karl Meinke, EECS School, KTH Kungliga Tekniska Högskolan, 100 44 Stockholm, Sweden. The date on the postmark will be taken as the date of your submission. The deadline of March 13th, 2025 applies to all manuscripts submitted by post.
- (4) If manuscripts are submitted in any other place or by any other means than those described in Part (3) then the examiner and EECS School cannot be held responsible in the case that manuscripts are lost. In the case of postal submission, it is strongly recommended that you keep a digital copy or photocopy in case of postal loss. KTH cannot be held responsible for loss in any national postal system.
- (5) If you have any questions about the exam (for example, if you do not understand a question) you may call the examiner on 08 790 6337. Please do not call before 10.00 am or after 5.00pm, and be aware that I may be in a meeting! You can also e-mail the examiner at karlm@kth.se.
- (6) I will publish any typographical errors and corrections that become known during the exam on the course exam web page and notify you with Canvas. Therefore, you should check back regularly on the course exam page for updates that will have new version numbers 1.x.
- (7) You may use your course notes, books and the internet. However, all material you submit must be your own. (a) You are not allowed to discuss your answers with anyone else until the exam is finished. (b) You are not allowed to copy anyone else's work. (c) By handing in your manuscript you are declaring that you have read and abide by all the rules on this cover page. (d) In the case that cheating is suspected, disciplinary action will be taken.
- (8) Hand-written answers must be written clearly. No marks will be awarded for work that I cannot read. You can write your answers in Swedish or in English. You can submit a hand-written manuscript or a typed manuscript or any combination of both.

NOTE: For a full score of 100 points you should answer all 4 questions below.

Question 1. (Total 35 points)

Write appropriate *pre*- and *post*-conditions for the following mathematical operations using the JML specification language (i.e. write *requires-ensures* conditions).

You should try to avoid both over and under-specification, i.e. write all and only the constraints necessary for each pre- and post condition. Do not omit relevant constraints or add irrelevant ones. **Hint: if you write a text comment next to each formula explaining its intended meaning then your work will be clearer and you will be less likely to make mistakes.**

If you are unsure how any mathematical operation/method is defined you may look it up online or in a book. You do not have to provide Java code for any of the operations, only JML.

Recall (or look up) that a *max-heap* is a labelled tree data structure that satisfies the following property. In a *max-heap*, for any given node c , if node p is the parent node of c (i.e. c is a child node of p) then the *key* (i.e. the value stored) at p is greater than or equal to the *key* at c . Note that a max-heap is not necessarily a complete tree, i.e. it can be unbalanced. The node at the top of the max-heap (with no parents) is called the *root* node. **Note:** the empty heap, with no keys at all, will be regarded as a valid max-heap.

In all questions below, we will assume that a *key* is a Java `Integer` object.

A common implementation of the max-heap uses a binary heap in which the tree is a binary tree. This can be simply and easily implemented using a linear array, in which each array element represents one node in the binary tree, and stores one element (the node key) of type `Integer`. The parent-child relationship is implicitly represented by the indexing of the array, so each node at index i has its parent node at index $\lfloor (i-1)/2 \rfloor$ (Here $\lfloor . \rfloor$ is the so-called “floor function”).

In all questions below, we will assume that a max-heap is implemented as a binary heap. **You should reuse your answer to part (i) to make your answers to parts (ii) and (iii) shorter.**

- (i) **(10 points)** The *max-heap property* for an *Integer* array object `inArray`

```
boolean isMaxHeap(Integer [] inArray)
```

Input: An `Integer` array `inArray`.

Output: `true` if, and only if, `inArray` represents a valid max-heap.

- (ii) **(5 points)** A `getRoot` function

```
Integer getRoot(Integer [] inArray)
```

Input: An `Integer` array `inArray` which is a valid max-heap.

Output: The `Integer` object which is the key at the root of the max-heap `inArray`.

- (iii) **(20 points)** A `deleteRoot` function

```
Integer [] deleteRoot(Integer [] inArray)
```

Input: An `Integer` array `inArray` which is a valid max-heap.

Output: A valid max-heap which is obtained from `inArray` by deleting the key at the root node.

Question 2. (Total 10 points)

- (i) **(4 points)** Write down four different metamorphic equations that should be satisfied by any test case for the *root mean square function (aka. RMS)*:

```
float rms(float[] sample)
```

Input: A one-dimensional array `sample` of `float` (the data set).

Output: The `float` result which should be the square root of the arithmetic mean of the squares of all `sample` values `sample[0]`, ..., `sample[n]`.

Note: For this example, you may assume that floating point operations are perfectly accurate.

Clearly define the data transformation

$$T: \text{float}[] \rightarrow \text{float}[]$$

used on the input array for each of your four different metamorphic equations.

- (ii) **(6 points)** For each of your four metamorphic equations e_i , $i = 1, \dots, 4$, derived for Part (i), consider its associated transformation T_i . Starting from one seed test case

$$x_0 : \text{float}[]$$

discuss whether the sequence of test cases

$$x_{j+1} = T_i(x_j) : j = 0, 1, 2, \dots$$

generated by iterating T_i repeats itself.

If all of your test case sequences (x_i) above are finite, does there exist a metamorphic equation e for `SetEquality` with a transformer T which can generate a test suite of $TS = \{ (x_0, y_0), \dots, (x_k, y_k) = T^k(x_0, y_0) \}$ of unlimited size by simply iterating the transformer T k -times?

Question 3. (Total 21 points)

Consider the UML state machine diagram for a *vehicle cruise controller* given in Figure 1 below.

A cruise controller is a real-time embedded application that interacts with the gas (acc) and brake (brake) pedal events, as well as the dashboard (off) and (resume) command events, to set and maintain a constant cruising speed (desiredSpeed), based on the vehicles current speed (speed). The application also interacts with global vehicle signals such as switching the engine on and off. All events, Boolean and floating-point variables in the diagram below should be clear, from this application description. You may assume that every action `Increase ( speed )` will increase the value of `speed` by a fixed constant `c` by the end of one time-unit, i.e.

`always ( speed == y & Increase(speed)  $\Rightarrow$  next(speed >= y+c) )`

While every action `Monitor ( speed )` updates the current value of `speed` according to the actual physical speed of the vehicle (given by some sensor such as wheel rotation) after one time-unit.

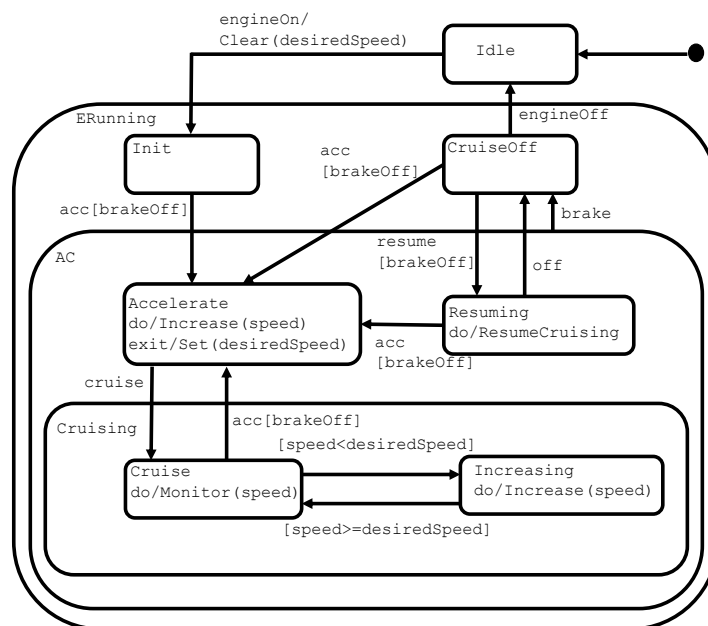


Figure 1. UML state machine for a cruise controller

- (i) **(15 points)** For each of the LTL formulas below, state whether it is true or false about the cruise controller model in Figure 1. If a formula is false, you should give a counterexample trace that demonstrates an execution which fails the property.

- a) `always( state = ERunning.CruiseOff`

- b) `always(state = ERunning.AC ⇒
state = ERunning.AC.Accelerate |
state = ERunning.AC.Resuming |
state = ERunning.AC.Cruising))`
 - c) `sometime(state = ERunning.AC.Cruising)`
 - d) `always(state = ERunning.AC.Cruising.Increasing & speed <= desiredSpeed ⇒
next(state = ERunning.AC.Cruising.Increasing))`
 - e) `always(state = ERunning.AC.Cruising.Increasing & brake ⇒
next(state=CruiseOff))`
 - f) `always(state = ERunning.AC.Cruising.Cruise & speed > desiredSpeed ⇒
next(speed <= desiredSpeed))`
- (ii) **(6 points)** A well-known safety hazard with cruise controller software is popularly known as *unintended acceleration*, when the vehicle can accelerate in an uncontrollable and dangerous way.
- a) Explain, how you could use software testing to uncover such an implementation error in a code implementation of the UML state machine diagram of Figure 1 above.
 - b) Comment on whether model-based testing using Figure 1 would generate such a test case.
 - c) Based on your answer to (b), what conclusions can you draw about model-based testing of embedded software?

Question 4. (Total 34 points)

Harry Hopeful wanted to rigorously unit test the following code

```

1. myCode(int x): int
2. {
3.     int y;
4.     x:= x*2 + y;
5.     while ( x>-1 ) do
6.     {
7.         if ((x mod 2 == 0) & x > 1 & x <= 2 |
8.         (x <= 2 & !( x mod 2 == 0) & !(x>1) |
9.         !(x>1) & (x mod 2 == 0)& x <= 2)
10.        then

```

```
11.         print("A");
12.     else
13.         print("B");
14.     x := x - 2;
15. } ;
16. return x;
18. }
```

- (a) (2 points) Draw a condensation graph for the routine myCode.
- (b) (8 points) Write out a full set of test requirements for the condensation graph of part (a) using the logic coverage model of *correlated active clause coverage* (CACC) (also known as *Masking MCDC*).
- (c) (18 points) Write out a full set of test cases to satisfy as many of your CACC requirements as possible. What is the maximum percentage of coverage that can be achieved for CACC requirements? What can you say about the masking effects of the program clauses on each other?
- (d) (6 points) Choose any one test requirement that cannot be satisfied in part (c) and using the method of symbolic evaluation write a constraint table which shows that the requirement cannot be satisfied.